

ADDRESSING MODES

(Immediate mode, Register mode, Absolute mode, Indirect mode, Index Mode,
Auto increment mode, Auto decrement mode)

Sem: 4th Section (s): COA-CSE-G1, COA-CSCE-G1

Faculty: Prof. Anil Kumar Swain

Lecturer Summary

1. Definition of addressing modes
2. Generic Addressing modes
 - Immediate mode
 - Register mode
 - Absolute (Direct) mode
 - Indirect mode
 - Index mode
 - Auto increment mode
 - Auto decrement mode

1 Introduction

- **Addressing is the process of locating the operand (specific information) for a given operation.** It is similar to the process of obtaining your address so that information can be sent to you. Once the computer knows **where** to obtain the location of the operand, the instruction can be carried out. If for instance, the operand is in memory, the addressing technique determines how to obtain the memory address of the operand and how to use this address to locate the operand and fetch it. If the operand is in one of the CPU's registers, addressing is the means by which the instruction specifies the selected register and the operand is fetched. Because the length of instructions and the number of bits per memory cell vary between types of instructions and computers, there is a variety of ways the operand may be obtained.
- In general, a program operates on data (operand) that reside in the computer's memory. These data can be organized in a variety of ways.
- **Defⁿ: The different ways in which the location of an operand (data) is specified in an instruction are referred to as addressing modes.**
- The strategy to identify WHERE the operand is located is called addressing mode. Each addressing mode has a unique formula to locate the operand.
- **Effective address** is the actual address or location of operand to be used by the instruction.

2 Generic Addressing Modes

Refer next page

Generic Addressing Modes

Name	Assembler syntax	Addressingfunction
Immediate	#Value	Operand = Value
Register	R_i	$EA = R_i$
Absolute (Direct)	LOC	$EA = LOC$
Indirect	(R_i) (LOC)	$EA = [R_i]$ $EA = [LOC]$
Index	$X(R_i)$	$EA = [R_i] + X$
Base with index	(R_i, R_j)	$EA = [R_i] + [R_j]$
Base with index and offset	$X(R_i, R_j)$	$EA = [R_i] + [R_j] + X$
Relative	$X(PC)$	$EA = [PC] + X$
Autoincrement	$(R_i) +$	$EA = [R_i]$; Increment R_i
Autodecrement	$-(R_i)$	Decrement R_i ; $EA = [R_i]$

2.1 IMPLEMENTATION OF VARIABLES AND CONSTANTS

a) Representation of Constants in Assembly Language

In assembly language, address and data constants can be represented using the immediate mode.

Immediate mode — In this mode of addressing, the operand is given explicitly as part of the instruction itself.

“Effective Address”: operand located in the INSTRUCTION

Example

Ex1. `MOVE #200, R0` places the value 200 in register R0

b) Representation of Variables in Assembly Language

In assembly language, a variable can be represented by using two addressing modes namely register mode and absolute mode.

Register mode — The operand is the contents of a processor register; the name (address) of the register is given in the instruction.

Example

`MOVE R2, LOC` loads the operand in R2 to location LOC

Absolute mode/Direct mode — The operand is in a memory location; the address of this location is given explicitly in the instruction. (In some assembly languages, this mode is called Direct.)

MOVE LOC,R2 loads the operand in LOC to register R2

Ex2. Write a program in assembly language to represent the expression $A = B + 6$, Written in C language.

Answer:

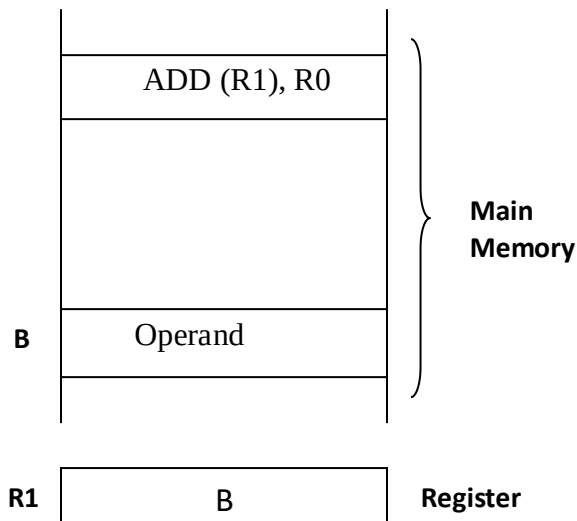
```
MOVE B, R1
ADD #6, R1
MOVE R1, A
```

2.2 INDIRECTION AND POINTERS

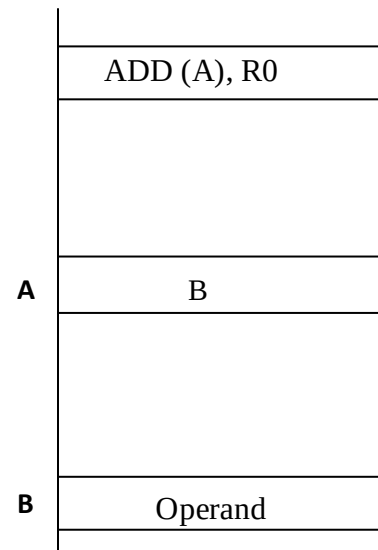
- **Indirect mode** — The effective address of the operand is the contents of a register or memory location whose address appears in the instruction.
- We denote indirection by placing the name of the register or the memory address given in the instruction in parentheses.

$(Ri) \Rightarrow EA = [Ri]$ $(A) \Rightarrow EA = [A]$

- The register or memory location that contains the address of an operand is called a **pointer**.



(a) Through a general-purpose register



(b) Through a memory location

• Examples

Sl.	Instructions	Meaning	Remarks
1	ADD (R1), R0	Contents of the location whose address is given in register R1 is added with the contents of register R0 and the result is placed in register R0.	$[R1] \Rightarrow B$ $[B] \Rightarrow \text{Operand}$ Here register R1 is known as the pointer. $EA = [R1]$

2	ADD (A), R0	Contents of the location whose address is given in location A is added with the contents of register R0 and the result is placed in register R0.	[A]=>B [B]->Operand Here location A is known as the pointer. EA = [A]
---	-------------	--	--

- Examples

Q1. Write down the C-language statement $A=*B$ in generic assembly language.

Answer: By Register Indirect Mode

MOVE B, R1
MOVE (R1), A

By Memory Indirect Mode

MOVE (B), A

Q2. Assume that a list of n numbers is stored in the memory of 32-bit word length of a byte addressable computer in successive locations NUM1, NUM2, NUM3, ..., NUMn respectively. The value of n is stored in memory location N. Write a program in assembly language to add the list of numbers and store the result in location SUM.

Answer:

Address	Contents	Remarks
LOOP	MOVE N,R1	Load the contents of location N to register R1
	MOVE #NUM1, R2	Load the address of first number into register R2
	CLEAR R0	Clears R0 to 0
	ADD (R2), R0	R2 is used as the pointer to the numbers in the list.
	ADD #4, R2	Reduces the contents of R1 by 1 each time thru a loop
	DECREMENT R1	
	BRANCH > 0 LOOP	
	MOVE R0, SUM	

Explanation

Indirect addressing can be used to access successive numbers in the list. The first time through the loop, the instruction ADD (R2), R0 fetches the operand at location NUM1 and adds it to R0. The second Add instruction adds 4 to the contents of the pointer R2, so that it will contain the address value NUM2 when the above instruction is executed in the second pass through the loop.

- BRANCH Instruction**

In the above program, the instruction

BRANCH > 0 LOOP

(branch if greater than 0) is a conditional branch instruction that causes a branch to location LOOP if the result of the immediately preceding instruction, which is the decremented value in register R1, is greater than zero. This means that the loop is repeated as long as there are entries in the list that are yet to be added to R0. At the end of the nth pass through the loop, the DECREMENT instruction produces a value of zero, and, hence, branching does not occur. Instead, the Move instruction is fetched and executed. It moves the final result from R0 into memory location SUM.

General use of BRANCH Instruction: This type of instruction loads a new value into the program counter. As a result, the processor fetches and executes the instruction at this

new address, called the *branch target*, instead of the instruction at the location that follows the branch instruction in sequential address order. A *conditional branch* instruction causes a branch only if a specified condition is satisfied. If the condition is not satisfied, the PC is incremented in the normal way, and the next instruction in sequential address order is fetched and executed.

2.3 INDEXING AND ARRAYS

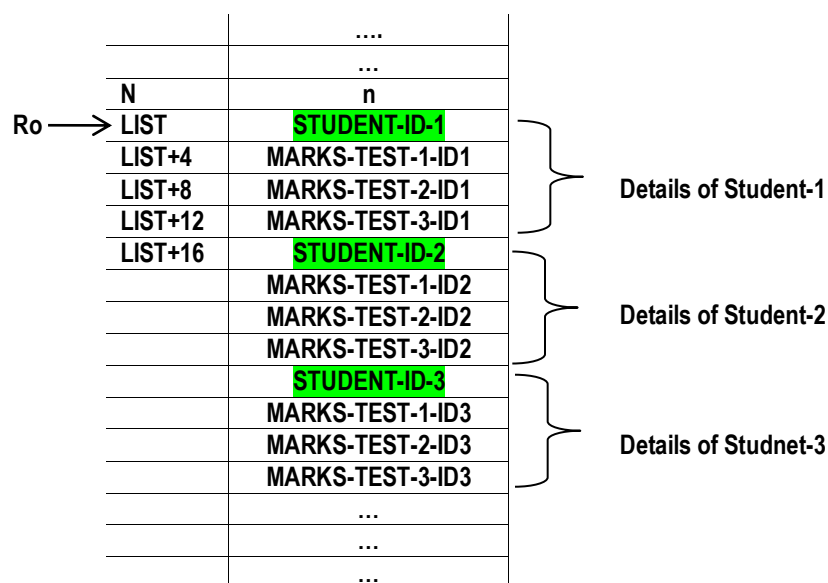
- It is useful in dealing with lists and arrays.
- **Index mode** — The effective address of the operand is generated by adding a constant value to the contents of a register. The constant value is called the offset.
- The register used may be either a special register provided for this purpose, or, more commonly, it may be any one of a set of general-purpose registers in the processor. In either case, it is referred to as an index register. We indicate the Index mode symbolically as **X(Ri)** where X denotes the constant value contained in the instruction and Ri is the name of the register involved. The effective address of the operand is given by

$$EA = X + [Ri]$$

- The contents of the index register are not changed in the process of generating the effective address. In an assembly language program, the constant X may be given either as an explicit number or as a symbolic name representing a numerical value.

- **Example**

Q1. Consider a list of test scores for students taking a given course. Assume that the list of scores, beginning at location LIST, is structured as shown in Figure below. A four-word memory block comprises a record that stores the relevant information for each student. Each record consists of the student's identification number (ID), followed by the scores the student earned on three tests. There are n students in the class, and the value n is stored in location N immediately in front of the list. The addresses given in the figure for the student IDs and test scores assume that the memory is byte addressable and that the word length is 32 bits. Write a program in assembly language to compute the sum of all scores obtained on each of the tests and store these three sums in memory locations SUM1, SUM2, and SUM3.



Answer:

Address	Contents	Remarks
LOOP	MOVE #LIST, R0	R0 is used as index register
	CLEAR R1	Clears R1 to 0
	CLEAR R2	Clears R2 to 0
	CLEAR R3	Clears R3 to 0
	MOVE N, R4	R4 is used as a counter
	ADD 4(R0), R1	
	ADD 8(R0), R2	
	ADD 12(R0), R3	
	ADD #16, R0	
	DECREMENT R4	
	BRANCH > 0 LOOP	
	MOVE R1, SUM1	
	MOVE R2, SUM2	
	MOVE R3, SUM3	

Explanation

- In the body of the loop, the program uses the Index addressing mode in the manner depicted in Figure given above to access each of the three scores in a student's record. Register R0 is used as the index register.
 - Before the loop is entered, R0 is set to point to the ID location of the first student record; thus, it contains the address LIST. On the first pass through the loop, test scores of the first student are added to the running sums held in registers R1, R2, and R3, which are initially cleared to 0. These scores are accessed using the Index addressing modes 4(R0), 8(R0), and 12(R0).
 - The index register R0 is then incremented by 16 to point to the ID location of the second student.
 - Register R4, initialized to contain the value n, is decremented by 1 at the end of each pass through the loop. When the contents of R4 reach 0, all student records have been accessed, and the loop terminates. Until then, the conditional branch instruction transfers control back to the start of the loop to process the next record.
 - The last three instructions transfer the accumulated sums from registers R1, R2, and R3, into memory locations SUM1, SUM2, and SUM3, respectively.
 - It should be emphasized that the contents of the index register, R0, are not changed when it is used in the Index addressing mode to access the scores. The contents of R0 are changed only by the last ADD instruction in the loop, to move from one student record to the next.
- **Basic form of indexed addressing**
Several variations of this basic form provide for very efficient access to memory operands in practical programming situations.
Type-1: (R_i, R_j) => EA = [R_i] + [R_j]
The second register is usually called the base register. In other words, this mode implements a two-dimensional array.
Type-2: X(R_i, R_j) => EA = X + [R_i] + [R_j]
In this case, the effective address is the sum of the constant X and the contents of registers R_i and R_j. This added flexibility is useful in accessing multiple components inside each

item in a record, where the beginning of an item is specified by the (R_i, R_j) part of the addressing mode. In other words, this mode implements a three-dimensional array.

2.4 RELATIVE ADDRESSING

- Index mode is defined by using general-purpose processor registers.
- A useful version of this mode is obtained if the program counter, PC, is used instead of a general purpose register.
- Instead of X(R_i), X(PC) can be used to address a memory location that is X bytes away from the location presently pointed to by the program counter. Since the addressed location is identified “relative” to the program counter, which always identifies the current execution point in a program, the name **Relative mode** is associated with this type of addressing.

$$X(PC) \Rightarrow EA = X + [PC]$$

2.5 ADDITIONAL MODES

The five basic addressing modes (Immediate, Register, Absolute (Direct), Indirect, and Index) are found in most computers. Although these modes suffice for general computation, many computers provide additional modes intended to aid certain programming tasks. The two modes described below are useful for accessing data items in successive locations in the memory.

Addressing Mode	Representation	Effective Address	Remarks
Autoincrement mode	(R _i) ⁺	EA = [R _i] R _i = [R _i] + 1	The effective address of the operand is the contents of a register specified in the instruction. After accessing the operand, the contents of this register are automatically incremented to point to the next item in a list.
Autodecrement mode	-(R _i)	R _i = [R _i] – 1 EA = [R _i]	The contents of a register specified in the instruction are first automatically decremented and are then used as the effective address of the operand.

N. **B.** Implicitly, the increment/decrement amount is 1 when the mode is given in the above form. But in a byte addressable memory, the increment/decrement is 1 for byte-sized operands, 2 for 16-bit operands, and 4 for 32-bit operands etc. This mode would only be useful in accessing successive words in a byte-addressable memory

2.6 HOME ASSIGNMENT

Question-1

Registers R1 and R2 of a computer contain the decimal values 1200 and 4600. What is the effective address of the memory operand in each of the following instructions? Assume the computer is 4-bytes byte addressable computer.

- Load 20(R1), R5
- Move #3000, R5
- Store R5, 30(R1, R2)
- Add -(R2), R5
- Subtract (R1)+, R5

Addressing Modes	Applications
Immediate Addressing Mode	To initialize registers to a constant value
Direct Addressing Mode and Register Direct Addressing Mode	- To access static data - To implement variables
Indirect Addressing Mode and Register Indirect Addressing Mode	- To implement pointers because pointers are memory locations that store the address of another variable - To pass array as a parameter because array name is the base address and pointer is needed to point the address
Relative Addressing Mode	- For program relocation at run time i.e. for position independent code - To change the normal sequence of execution of instructions - For branch type instructions since it directly updates the program counter
Index Addressing Mode	- For array implementation or array addressing - For records implementation
Base Register Addressing Mode	- For writing relocatable code i.e. for relocation of program in memory even at run time - For handling recursive procedures
Auto-increment Addressing Mode and Auto-decrement Addressing Mode	- For implementing loops - For stepping through arrays in a loop - For implementing a stack as push and pop